Plan de trabajo · Evaluación 2

Diagnóstico de rodamientos mediante MDE, reducción de dimensionalidad y regresión Softmax.

Preparado el 19 de octubre de 2026. Estado: planificación; no se han procesado los datos ni entrenado modelos. Entrega indicada en el enunciado: sábado 24 de octubre de 2026, 23:00, aula virtual del curso. La zona horaria de esa entrega no se especifica en el PDF.

1. Alcance y fuentes de verdad

El resultado buscado es una aplicación reproducible que clasifique cuatro estados de salud de un rodamiento a partir de señales de vibración. Cada ventana se representa mediante características MDE; estas se reducen con SLS y después con PCA-SVD; finalmente, Softmax estima la clase.

Este documento organiza el trabajo y las comprobaciones. No constituye todavía la solución de la evaluación ni presenta resultados experimentales.

Fuentes revisadas y trazabilidad

- **[E] Enunciado-Evaluacion2.pdf, 10 páginas.** Objetivos: p. 2; diagrama: pp. 3-4; archivos y tareas: pp. 5-8; entrega: p. 9.
- **[P] Integral-PCA-NonLinearPCA-RFF+RLS.pdf, 38 páginas.** PCA-SVD: pp. 3-12; RFF: pp. 14-21; SLS: pp. 23-31; entrenamiento por minibatches: pp. 33-36.
- **[S] SOFTMAX.pdf, 18 páginas.** Etiquetas: pp. 2-4; modelo: pp. 5-6; entropía cruzada: p. 10; RMSprop: pp. 12-15; métricas: p. 16.
- **[M] MultEnt.pdf, 24 páginas.** MDE: pp. 2-7; MPE: pp. 8-12; eMDE: pp. 14-18; eMPE: pp. 19-23. Para esta evaluación se utiliza MDE, según [E], p. 2.

Las páginas citadas corresponden a la posición de la página dentro de cada PDF. Se revisaron el texto y las páginas renderizadas, incluidos diagramas y fórmulas.

Criterio de trabajo: [E] determina los entregables y el método solicitado para esta evaluación; [M], [P] y [S] aportan su desarrollo matemático. Las diferencias entre ellos quedan registradas para consulta. Las recomendaciones de este plan se identifican como tales y no se atribuyen al profesor.

Cadena que se implementará

```
CSV de 4 clases -> ventanas de W muestras -> características MDE
-> separación training/testing y normalización con training
-> SLS: H = f(X V), reconstrucción Y = H U
-> PCA-SVD de H -> Xrr
-> normalización de Xrr con training -> Softmax -> clase y métricas
```

Correspondencia de scripts: `mdef.py` prepara datos; `dimr.py` aprende la reducción; `train.py` aprende Softmax; `test.py` aplica los modelos guardados y evalúa. El orden obligatorio de dependencia es `mdef.py -> dimr.py -> train.py -> test.py`.

Por qué no se sustituye por RFF aleatorio: [P] presenta varias alternativas y [S], p. 17, menciona PCA+RFF. Sin embargo, [E], pp. 2 y 6, pide pesos no lineales aprendidos por RMSprop, pesos lineales por RLS y una etapa posterior de PCA. Esa es la cadena principal del plan; una comparación con RFF sería adicional.

2. Decisiones y diferencias que hay que resolver

Estos puntos no deben completarse inventando requisitos. Se puede avanzar en interfaces, trazabilidad y verificaciones mientras se obtiene la información.

1. **Datos y cuatro clases.** [E], p. 5, menciona CSV de cuatro clases, pero no entrega nombres, rutas, enlace de descarga, canal, frecuencia de muestreo ni correspondencia clase-archivo. Registrar esa información al recibir la carpeta de datos; no asumir un dataset conocido de rodamientos.
2. **MDE: estructura definida, función interna pendiente.** [M], pp. 4-5, especifica el promedio por escala y `MDE(τ) = Entropy(y^(τ), d, tau, c)`. Su p. 7 muestra d=3, tau=1, c=3 y Smax=4 como ejemplo, no como obligación para estos datos. No desarrolla el mapeo a símbolos, conteo de patrones, base del logaritmo ni normalización dentro de `Entropy`. Confirmar esa rutina del curso y los valores a usar; no deducirlos solo de la gráfica.
3. **Ventanas y partición.** [E] pide tamaño W y un porcentaje training/testing, pero no fija valores, solapamiento ni política de sobrantes. Elegirlos y documentarlos antes de evaluar. El 80/20, por ejemplo, sería una propuesta, no una exigencia textual.
4. **Activación y optimizador SLS.** [P], p. 24, deja `f` como función no lineal sin concretarla; pp. 26 y 30 usan mGD. [E] exige RMSprop. Mantener el modelo y gradiente SLS y usar RMSprop por indicación explícita de [E]. Confirmar `f` y su derivada; no imponer coseno, tanh o ReLU como si estuvieran prescritos.
5. **Salida no lineal.** [P], p. 31, escribe `Xr = X V`; [E], p. 6, exige `Xr = f(X V)`. Para esta evaluación se usará esta última expresión. Omitir `f` cambiaría la representación que entra a PCA.
6. **Umbral PCA.** [P], p. 11, usa acumulación <= 95% y elige cuatro componentes con 93.0828%. Retener al menos 95% requeriría cinco, con 99.0177%. Consultar cuál criterio se evaluará; conservar la curva completa y el criterio elegido. No cambiar silenciosamente <= por >=.
7. **Normalización y testing.** [E], p. 5, enumera normalizar antes de dividir. Recomendación matemática: fijar primero los índices de separación y ajustar estadísticas solo con training. Es una modificación explícita del orden literal para evitar fuga de información. En testing se reutilizan esas estadísticas; no se estiman de nuevo.
8. **Persistencia y carga.** [E] no especifica formato interno ni extensión para `dtrain`, `dtest`, `pesos`, `pesoftmax` y `costo`. Conservar esos nombres base y acordar su serialización. Aunque p. 8 no lo enumera, `test.py` necesita también `pesoftmax` y las estadísticas de normalización.
9. **Factores y orientación.** [P], p. 28, incluye dos factores 1/2 en su costo anidado; las fórmulas de gradiente de ambos apuntes no mantienen una orientación matricial uniforme. Escribir costo, dimensiones y gradiente juntos antes de programar; comprobar su coherencia numéricamente. Ver sección 6.

Checklist de acuerdos previos

[ ] Obtener ruta y descripción de los CSV y las cuatro clases.

[ ] Confirmar la rutina interna `Entropy`, parámetros MDE y `f` con su derivada.

[ ] Registrar W, paso, partición, normalizaciones, J, criterio PCA y formatos.

[ ] Resolver o dejar identificadas las consultas docentes; no declarar resueltas las discrepancias.

3. Parte 0: inventario y contrato de datos

Dónde se aplica: antes de `mdef.py`. **Responsable sugerido:** persona de datos, con revisión cruzada. **Dependencias:** acceso a los CSV. **Salida:** inventario y configuración documentados; son apoyos recomendados, no nombres exigidos por [E].

Registrar para cada archivo: clase real, orden de su etiqueta, número de muestras, columnas, unidades, canal usado, frecuencia de muestreo si está disponible, identificador de adquisición y condiciones de operación. Conservar los datos originales. Revisar separadores, encabezados, valores ausentes, valores no finitos y posibles duplicados.

Definir un único diccionario de clases. Para cuatro clases, los códigos son `[1,0,0,0]`, `[0,1,0,0]`, `[0,0,1,0]` y `[0,0,0,1]`, según [S], pp. 3-4. Los nombres reales se completarán con los datos; no están definidos en los PDF.

Ficha que el equipo debe completar antes de experimentar

Campo	Qué se debe registrar
Datos originales	Carpeta, archivos, canal, unidades y clases reales.
Procedencia	Adquisición y condición operativa; grupo de cada registro.
Ventanas	W, paso s, solapamiento y política del tramo incompleto.
MDE	d, tau de retardo, c, Smax y versión de la rutina Entropy.
Separación	Porcentaje, grupos o bloques, semilla e índices guardados.
Normalización	Método, desviación estándar y columnas constantes.
SLS	J, f, f', lambda, inicialización y convención de costo.
RMSprop	Tasas, beta, epsilon, tamaño de batch y épocas por modelo.
PCA	Dimensión r, criterio y porcentaje real de energía retenida.
Softmax	Orden de clases, sesgo, normalización reducida y parada.
Persistencia	Formatos de dtrain, dtest, pesos, pesoftmax y costo.

Esta ficha es una propuesta organizativa. Los campos sin valor no son valores predeterminados: deben cerrarse con datos, material del curso o decisiones justificadas del equipo. Los parámetros del ejemplo MDE no se copian automáticamente como parámetros finales de la evaluación.

Checklist de la parte 0

[ ] Identificar todos los CSV y vincularlos a exactamente cuatro clases.

[ ] Definir columna de señal, unidades, tratamiento de datos inválidos y orden de clases.

[ ] Asignar identificadores de adquisición y conservar la procedencia de cada ventana.

[ ] Registrar configuración y semillas; verificar que ninguna etiqueta se use como característica.

4. Parte 1: ventanas y características MDE · mdef.py

Fuentes: [E], pp. 2 y 5; [M], pp. 3-7. **Entrada:** señales de vibración. **Salida intermedia:** una fila de características por ventana y su etiqueta. Hay dos niveles distintos: ventana externa de W muestras para clasificación y bloques internos de τ muestras para cada escala.

Segmentación externa

Segmentar cada señal sin mezclar clases ni adquisiciones. El paso s determina el solapamiento. Para una señal de L muestras y descartando el sobrante, hay `max(0, 1 + floor((L-W)/s))` ventanas, con W y s enteros positivos. Registrar archivo, intervalo de muestras, clase y grupo de cada ventana. Fijar qué ocurre con el tramo final incompleto; no rellenar con ceros sin justificarlo.

Fórmulas de MDE que sí están en el apunte

Aplicar [M], p. 4, dentro de cada ventana externa, cuyo largo es W (el apunte llama N al largo de la serie). Para cada escala entera τ entre 1 y Smax:

```
y_j^(τ) = (1/τ) sum_(i=(j-1)τ+1..jτ) x_i
1 <= j <= floor(W/τ)
MDE(τ) = Entropy(y^(τ), d, tau, c)       [M], p. 5
F_n = [MDE_n(1), MDE_n(2), ..., MDE_n(Smax)]
```

τ es la escala; `tau`, escrito como argumento de `Entropy`, es el retardo de embedding. No son el mismo parámetro. `d` es la dimensión de embedding, `c` el número de símbolos y Smax la escala máxima. Para un canal y una entropía por escala, cada ventana produce D=Smax características. D será la dimensión de entrada de SLS; no confundirla con d de embedding ni con r de PCA.

Cada escala se calcula desde la ventana original, con bloques internos completos y no solapados de τ muestras. No aplicar sucesivamente el promedio de escala 2 sobre el resultado de la escala 1 para construir todas las escalas. A escala 1 se recupera la serie original y su entropía de dispersión.

Parámetros y verificaciones que faltan concretar

El ejemplo de [M], p. 7, usa d=3, tau=1, c=3 y Smax=4. Se puede usar como configuración de referencia, pero no imponerla al caso real sin acordarlo. Confirmar el cálculo interno de `Entropy`: mapeo a símbolos, patrones, probabilidades, logaritmo y normalización. Si utiliza estadísticas, precisar si se estiman por ventana, por serie original o sobre training, según su definición.

Con embedding convencional, el número de patrones disponibles sería `floor(W/τ) - (d-1)*tau`; debe ser al menos 1 y, en la práctica, suficiente para estimar frecuencias. Esta condición es un control derivado de la definición de embedding, que se contrastará con la rutina acordada; no garantiza por sí sola una estimación fiable. El máximo de patrones simbólicos posibles es c^d bajo ese esquema.

No sustituir variantes: MPE usa patrones de orden; eMDE promedia entropías de varias subseries desplazadas [M], pp. 15-16. Ninguna reemplaza la MDE solicitada. Tampoco se debe exigir que la entropía disminuya siempre con la escala: la gráfica de ejemplo no es una ley de monotonía.

Checklist de la parte 1

[ ] Fijar W, paso, sobrantes, d, tau, c y Smax; distinguir escala de retardo.

[ ] Implementar las fórmulas de [M], pp. 4-5, y confirmar la rutina `Entropy`.

[ ] Revisar a mano, por ejemplo, que [1,3,5,7,9] a escala 2 da [2,6].

[ ] Mantener una etiqueta y una procedencia por cada fila de características.

[ ] Revisar casos constantes, longitud por escala y suficientes patrones según la rutina.

[ ] Confirmar D=Smax para un canal, valores finitos y columnas en orden de escala.

5. Parte 2: apilado, separación y normalización · mdef.py

Fuente: [E], p. 5; normalización z-score en [P], p. 14. **Entrada:** ventanas ya representadas por MDE. **Salida exigida:** `dfeatures.csv`, `dclass.csv`, `dtrain`, `dtest`.

Procedimiento propuesto

1. Apilar las características de las cuatro clases en una matriz de tamaño N x D y las etiquetas one-hot en otra de tamaño N x 4. Mantener un índice de procedencia separado.
2. Generar una única permutación aleatoria y aplicarla a las mismas filas de ambas matrices y sus metadatos. Las dos instrucciones de reordenamiento de [E] no deben implementarse con sorteos independientes.
3. Fijar los índices de training/testing antes de estimar estadísticas. Procurar presencia de las cuatro clases en ambos conjuntos. Si hay varias adquisiciones, separar por adquisición; si solo hay una señal por clase, proponer bloques temporales separados y declarar la limitación para generalizar.
4. Evitar que ventanas solapadas compartan muestras entre conjuntos. Para señales continuas, seleccionar bloques separados y generar o seleccionar ventanas contenidas en ellos; cuando corresponda, dejar una separación temporal. Un shuffle por sí solo no elimina esa dependencia.
5. Ajustar la normalización de las características únicamente en training. Propuesta: z-score por columna, respaldado por [P], p. 14. Aplicar los mismos parámetros a testing y guardar esas estadísticas. Registrar la convención de desviación estándar, por ejemplo `ddof=1`, sin atribuirla como exigencia de [E].
6. Guardar `dfeatures.csv` normalizado y `dclass.csv` con filas alineadas, más `dtrain` y `dtest` con sus características y etiquetas. La normalización de todas las filas usa parámetros de training, aunque se materialice un CSV conjunto.

```
mu_j = media de la característica j en training
sigma_j = desviación estándar de j en training
X[n,j] = (F[n,j] - mu_j) / sigma_j
T: matriz N x 4; T[n,k] pertenece a {0,1}; sum_k T[n,k] = 1
```

Si una columna tiene desviación cero, fijar una política guardada y consistente: por ejemplo, usar divisor 1 para esa columna. No dividir por cero ni elegir una política diferente en testing. Una característica constante en training no aporta variación para aprender; registrar su existencia.

Checklist de la parte 2

[ ] Crear exactamente los cuatro nombres exigidos y documentar su contenido.

[ ] Comprobar N filas de features, N etiquetas y suma one-hot igual a 1.

[ ] Aplicar una misma permutación a datos, etiquetas y procedencia.

[ ] Verificar separación de grupos o bloques y ausencia de muestras compartidas.

[ ] Guardar índices, semilla, estadísticas y tamaños por clase de cada conjunto.

[ ] Confirmar que ninguna estadística de ajuste proviene de testing.

Aceptación: cada fila puede rastrearse hasta su señal, y los archivos permiten reconstruir exactamente la partición y la transformación usada.

6. Parte 3: reducción no lineal SLS · dimr.py

Fuentes: [E], p. 6; [P], pp. 23-30 y 35-36. **Entrada:** X de `dtrain`, con Ntr filas y D columnas. **Salida de esta etapa:** V aprendida y `Xr = H`. Las etiquetas no son el objetivo de reconstrucción.

Modelo y pesos

```
V: D x J       Z = X V: Ntr x J
H = f(Z): Ntr x J
U: J x D       Y = H U: Ntr x D
E_rec = (1/2) ||H U - X||_F^2 + (lambda/2) ||U||_F^2
U = (H^T H + lambda I_J)^(-1) H^T X
```

Es el costo penalizado de [P], p. 26, expresado con norma de Frobenius para matrices. RLS significa aquí **Regularized Least Squares**, según p. 20; no un filtro de mínimos cuadrados recursivos. U reconstruye D variables desde J activaciones; no es la matriz de componentes principales del PCA posterior.

Para el costo anterior, manteniendo U fijo y usando muestras por filas, el gradiente coherente es:

```
G_V = X^T [ ((H U - X) U^T) .* f'(X V) ]      tamaño D x J
```

`.*` significa producto elemento a elemento. Esta expresión desarrolla la regla de la cadena de [P], p. 29, con dimensiones explícitas. Es una derivación para implementar y verificar, no una transcripción literal de su disposición gráfica.

Se presupone una activación f aplicada elemento a elemento. Elegir J compatible con la reducción buscada y r no mayor que el rango disponible del PCA. Registrar las dimensiones finales D -> J -> r; [E] no fija sus valores.

Convención de costo pendiente de acuerdo: [P], p. 28, define `E = (1/(2N)) sum_n E_n`, con `E_n = (1/2) sum_m error_nm^2`. Leída literalmente, la parte residual equivale a `||Y-X||_F^2/(4N)` y su gradiente, con U fijo, lleva `1/(2N)`. No corresponde usar ese costo con un gradiente sin tal factor. Si se promedia también el costo penalizado de p. 26 por el tamaño del batch, dividir todo ese costo y su gradiente por el mismo tamaño mantiene la solución RLS. Acordar y registrar una convención única; no mezclar pérdidas y gradientes de escalas diferentes.

Bucle de aprendizaje

Inicializar V y un acumulador RMSprop de su misma forma. Por época, reordenar training; por minibatch, calcular H, resolver U por RLS, calcular el gradiente respecto de V con U fijo y actualizar V con RMSprop. Usar las ecuaciones RMSprop de la sección 8, en lugar del mGD de [P], porque [E] lo exige. Tras el último cambio de V, recalcular H y, para dejar consistente la reconstrucción final, U con todo training.

Resolver el sistema lineal de RLS con un solucionador numérico, evitando formar la inversa explícita: es la misma ecuación matemática. Elegir lambda positivo favorece una solución única; su valor no está fijado en los PDF.

Checklist de la parte 3

[ ] Definir J, f, f', lambda, inicialización, batches, épocas y tasa de aprendizaje.

[ ] Acordar costo y factores; comprobar forma D x J del gradiente y diferencias finitas con U fijo.

[ ] Alternar U por RLS y V por RMSprop; conservar el acumulador entre actualizaciones.

[ ] Tratar explícitamente el último batch incompleto y usar su tamaño real si se promedia.

[ ] Obtener `Xr = f(X V)` y registrar convergencia de reconstrucción como control recomendado.

7. Parte 4: PCA lineal por SVD · dimr.py

Fuentes: [E], p. 6; [P], pp. 3-12. **Entrada:** `Xr = f(X V)` de training, tamaño Ntr x J. **Salida exigida:** archivo `pesos` y representación final `Xrr` de tamaño Ntr x r.

Procedimiento y fórmulas

1. Calcular la media de cada columna de Xr usando training y centrar esa matriz. Conservar la media para aplicar la misma transformación a nuevas entradas.
2. Para obtener la SVD con la normalización de [P], p. 4, usar la matriz centrada dividida por la raíz de Ntr-1. Este escalado global no es otro z-score por característica.
3. Obtener los vectores singulares derechos y ordenar por valor singular descendente. Si la biblioteca devuelve `Vh`, los vectores derechos son las columnas de `Vh.T`.
4. Seleccionar r según el criterio acordado. Guardar la curva de varianza explicada y la acumulada; el umbral de 95% del ejemplo no está establecido como requisito en [E].
5. Proyectar los datos centrados sobre las r columnas seleccionadas. La fórmula `Xrr = Xr Vpca` de [E] se interpreta después de la operación de centrado establecida por [P], p. 3; guardar ese centrado evita omitirlo en testing.

```
mu_H = mean_train(H, por columna)
Hc = H - mu_H
A = Hc / sqrt(Ntr - 1)
A = U_svd S V_svd^T
lambda_i = s_i^2
VE_i (%) = 100 s_i^2 / sum_j s_j^2
Accum_r (%) = sum_(i=1..r) VE_i
Vpca = V_svd[:, 1:r]           tamaño J x r
Xrr = Hc Vpca                  tamaño Ntr x r
```

El denominador de VE recorre todos los valores singulares disponibles, no solo los retenidos. Los ceros que falten en una SVD económica no cambian la suma. No calcular varianza explicada con `s_i / sum(s)`.

Criterio de retención que debe quedar escrito

En el ejemplo de [P], p. 11, cuatro componentes retienen 93.0828% y cinco, 99.0177%. La elección literal mostrada es cuatro con acumulación <=95%. Si se solicita conservar al menos 95%, la elección es cinco. Reportar el porcentaje realmente conservado; no afirmar que cuatro retienen al menos 95%.

Checklist de la parte 4

[ ] Centrar con training y guardar mu_H; comprobar que Ntr > 1.

[ ] Usar vectores singulares derechos y energía proporcional a s al cuadrado.

[ ] Registrar r, criterio de selección y varianza acumulada real.

[ ] Verificar `Vpca.T @ Vpca` aproximadamente igual a la identidad.

[ ] Guardar en `pesos` V, activación, Vpca, mu_H y dimensiones.

[ ] Guardar o referenciar en `pesos` la normalización de entrada y el orden MDE.

Aceptación: training y testing pueden recorrer la misma transformación guardada sin recalcular SVD. U y los diagnósticos de reconstrucción son apoyos recomendados para auditoría; U no sustituye a V ni a Vpca en la inferencia.

8. Parte 5: entrenamiento Softmax · train.py

Fuentes: [E], p. 7; [S], pp. 5-6, 10 y 12-15. **Entradas:** `dtrain`, `pesos`. **Salidas exigidas:** `pesoftmax`, `costo` y gráfica de costo.

Reconstruir la representación final con los pesos SLS y PCA guardados. [E], pp. 7-8, vuelve a llamarla Xr; en este plan se mantiene Xrr para distinguirla de la salida SLS. El clasificador usará la reducción completa de p. 6.

Normalizar Xrr ajustando nuevas estadísticas solo en training; conservarlas para testing. Como propuesta, usar z-score por columna. No confundir estas estadísticas con las de MDE ni con la media del PCA. Si se incluye intercepto, agregar una columna de unos después de normalizar; [S], p. 5, contempla el término de sesgo. Registrar esta elección.

Softmax y entropía cruzada

Para M filas del batch, llamar Q a la entrada normalizada del clasificador (incluido el sesgo si se usa), T a sus etiquetas y Wsm a sus pesos. Se usa una orientación con muestras por filas en todas las etapas.

```
Q: M x q       Wsm: q x 4       Zsm = Q Wsm: M x 4
P[n,k] = exp(Zsm[n,k]) / sum_(i=1..4) exp(Zsm[n,i])
E_ce = -(1/M) sum_(n=1..M) sum_(k=1..4) T[n,k] ln(P[n,k])
G_sm = Q^T (P - T) / M          tamaño q x 4
```

El gradiente es la forma equivalente por filas del gradiente de [S], pp. 11 y 15; no copiar sus productos sin adaptar transpuestas. Usar logaritmo natural para ser coherente con el gradiente mostrado.

RMSprop, también usado para V en SLS

```
A_(t+1) = beta A_t + (1-beta) (G_t .* G_t)
Theta_(t+1) = Theta_t - mu G_t / (sqrt(A_(t+1)) + epsilon)
A_0 = ceros de la misma forma que Theta
```

Son operaciones elemento a elemento; Theta es Wsm o V, con acumuladores independientes. [S], p. 15, fija beta=0.9 y ubica epsilon fuera de la raíz. La diapositiva imprime los extremos de epsilon en orden inverso; registrar un valor positivo, por ejemplo 1e-8, como decisión de implementación pendiente de ratificación, no como valor único exigido. La tasa mu pertenece a (0,1) en las diapositivas; se selecciona sin mirar testing.

Checklist de la parte 5

[ ] Cargar `dtrain` y `pesos`, y aplicar SLS, centrado PCA y proyección completos.

[ ] Guardar normalización reducida, orden de clases, uso de sesgo y Wsm en `pesoftmax`.

[ ] Reordenar Q y T juntos en cada época; procesar y ponderar correctamente los batches.

[ ] Calcular Softmax estable restando el máximo por fila y costo con log-sum-exp equivalente.

[ ] Verificar gradiente por diferencias finitas y que las probabilidades suman 1 por fila.

[ ] Guardar `costo` y su gráfica con ejes, definición del costo y número de época.

Para comparar épocas, se recomienda evaluar el costo medio con todos los datos de training y los pesos al cierre de cada época. Una curva por minibatches puede fluctuar; no exigir descenso monótono en cada actualización.

Si se ajustan hiperparámetros, reservar validación dentro de `dtrain`, respetando grupos o bloques. En esa comparación, ajustar normalizaciones, SLS y PCA solo con el subconjunto de entrenamiento interno. Una vez elegida la configuración, se puede reajustar con todo `dtrain` y medir una vez sobre `dtest`. Esta validación es una recomendación, no un archivo adicional exigido.

9. Parte 6: evaluación final · test.py

Fuentes: [E], p. 8; [S], pp. 6 y 16. **Entradas necesarias:** `dtest`, `pesos` y `pesoftmax`. **Salidas exigidas como resultados:** matriz de confusión y F-scores. El enunciado no fija nombres de archivos para estas salidas.

Procedimiento

1. Cargar las características de testing, sus etiquetas y los modelos persistidos. Si `dtest` ya contiene MDE normalizado, no normalizar esa entrada una segunda vez.
2. Calcular `Htest = f(Xtest V)`, restar mu_H de training y proyectar con Vpca. Normalizar la representación reducida con las estadísticas guardadas en `pesoftmax`. Agregar el mismo sesgo del entrenamiento si corresponde.
3. Calcular scores y Softmax; elegir la clase por `argmax` y traducir su posición al mismo diccionario de clases.
4. Comparar con las etiquetas reales y generar matriz de confusión y métricas. Evaluar el modelo final sin ajustar parámetros con estos resultados.

Convención de matriz de confusión de [S], p. 16

Las fórmulas de precisión por suma de fila y recall por suma de columna son coherentes si **las filas son las clases predichas y las columnas las reales**. Rotular ambos ejes con palabras y nombres de las clases.

```
CM[i,j] = número de casos predichos como i cuya clase real es j
Precision_i = CM[i,i] / sum_j CM[i,j]
Recall_i    = CM[i,i] / sum_j CM[j,i]
F1_i = 2 Precision_i Recall_i / (Precision_i + Recall_i)
Accuracy = trace(CM) / Ntest
```

Si una biblioteca devuelve filas reales y columnas predichas, transponer su matriz o adaptar los denominadores. No intercambiar las interpretaciones de precisión y recall.

Reportar F1 para cada una de las cuatro clases. Como complemento recomendado, incluir soporte por clase, precisión, recall, F1 macro y accuracy; [E] no exige un umbral mínimo ni un promedio concreto. Definir y declarar la política cuando un denominador es cero, por ejemplo devolver 0 e indicarlo.

Checklist de la parte 6

[ ] Cargar también `pesoftmax`, aunque no aparezca en la lista de carga de [E], p. 8.

[ ] Reutilizar V, f, media PCA, Vpca y ambas normalizaciones ya aprendidas.

[ ] Confirmar que no se ejecutan fit, actualización de pesos, RLS o SVD sobre testing.

[ ] Verificar matriz 4 x 4 y suma de sus elementos igual a Ntest.

[ ] Mostrar F1 por clase y ejes explícitos; revisar manualmente un ejemplo de conteos.

[ ] Conservar probabilidades, clases predichas y etiquetas para poder auditar los resultados.

Aceptación: `test.py` puede ejecutarse en un proceso nuevo usando únicamente datos y artefactos guardados. No depende de variables que quedaron en memoria durante training.

10. Revisión de corrección matemática

Estos controles son recomendaciones de revisión, no entregables adicionales impuestos por el enunciado. La revisión debe comprobar el modelo aplicado y sus derivadas, además de mirar las métricas finales.

Controles por bloque

- **MDE:** verificar promedios de bloques y longitudes floor(W/τ), sin mezclar τ con el retardo. Contrastar `Entropy` con un ejemplo oficial; la gráfica de [M], p. 7, no aporta una serie tabulada y salidas exactas para una prueba numérica completa. Verificar probabilidades y límites de entropía según la rutina confirmada, incluidos varianza nula, probabilidades cero y falta de patrones. No exigir monotonía con la escala.
- **Datos:** verificar que las etiquetas siguen a las filas tras cada shuffle. Buscar ventanas duplicadas o con muestras compartidas entre training/testing. Documentar si el experimento estima rendimiento sobre nuevas ventanas de una misma señal o sobre adquisiciones independientes.
- **RLS:** comprobar el residuo de las ecuaciones normales `(H.T H + lambda I) U - H.T X`, relativo a la escala del problema. Vigilar condicionamiento y ausencia de valores no finitos. Para el mismo H, U obtenido por RLS debe minimizar el costo cuadrático penalizado respecto de U.
- **SLS:** comprobar por diferencias finitas unas entradas de V contra el costo declarado y con U fijo. Si se prueba el costo reducido que vuelve a resolver U para cada V perturbada, incluir el término regularizador y resolver U con suficiente precisión. No comparar ese costo con la derivada de un objetivo distinto.
- **PCA:** comprobar media centrada cercana a cero, ortonormalidad de Vpca y covarianza aproximadamente diagonal de los scores de training. La energía acumulada debe ser no decreciente. El error de reconstrucción por proyección centrada no debería aumentar al conservar más componentes del mismo PCA.
- **Softmax:** para cuatro scores iguales debe obtenerse `[0.25,0.25,0.25,0.25]`. Sumar una constante a todos los scores de una fila no debe cambiar sus probabilidades. Un predictor uniforme tiene entropía cruzada `ln(4)`, aproximadamente 1.386294, como referencia de escala, no como garantía para cualquier inicialización.
- **RMSprop:** probar una actualización pequeña a mano; comprobar acumulador no negativo, signos, dimensiones, cuadrado elemento a elemento y epsilon fuera de la raíz. No confundir V de SLS con el acumulador que [S] también llama V.
- **Métricas:** con una matriz diagonal perfecta, precisión, recall y F1 son 1 para clases presentes. Con una matriz pequeña no simétrica, verificar manualmente los denominadores para detectar transposiciones.

Una discrepancia numérica verificada en SOFTMAX, p. 7

La diapositiva anuncia dos atributos, pero imprime un vector x de longitud tres y W de 3 x 3. Usando literalmente esos números:

```
x W = [0.23169276, 1.00576065, 0.62006011]
Softmax(x W) = [0.21537144, 0.46704801, 0.31758056]
```

Los scores mostrados allí son `[-0.2629, 0.5650, -0.8610]`; su Softmax sí produce aproximadamente `[0.2605, 0.5962, 0.1433]`. Por tanto, el desacuerdo está entre las entradas/pesos impresos y los scores. No usar todo el ejemplo como una prueba numérica coherente ni forzar el código a reproducir ambos resultados a la vez. Conservar la fórmula general y consultar la errata.

Checklist de revisión

[ ] Una segunda persona revisa dimensiones, costos, factores y derivadas.

[ ] Registrar tolerancias numéricas y errores relativos; no exigir igualdad exacta de coma flotante.

[ ] Revisar resultados altos o bajos a partir de datos, partición y transformaciones.

[ ] Separar mejoras elegidas con validación interna de la medición final en testing.

11. Contrato de archivos y resultado final

Los nombres siguientes son los del enunciado. No se cambian `pesoftmax` por otra grafía ni se agregan extensiones por suposición. El formato de los archivos sin extensión deberá acordarse; si se usa una biblioteca que añade extensiones automáticamente, controlar ese comportamiento.

Scripts obligatorios

- `mdef.py`: lee CSV, segmenta, obtiene MDE, crea etiquetas y prepara los datos.
- `dimr.py`: carga training, ajusta SLS con RMSprop+RLS y PCA-SVD y guarda la reducción.
- `train.py`: carga training y reducción, normaliza la representación final, entrena Softmax con RMSprop y guarda costo y pesos.
- `test.py`: aplica la cadena guardada a testing y obtiene matriz de confusión y F-scores.

Datos y modelos exigidos

- `dfeatures.csv`: filas MDE apiladas, reordenadas y normalizadas; cada fila corresponde a una fila de etiquetas.
- `dclass.csv`: etiquetas one-hot N x 4, con el mismo orden de filas y diccionario de clases.
- `dtrain`: características y etiquetas de entrenamiento, o una estructura que identifique ambas inequívocamente.
- `dtest`: características y etiquetas de prueba, con el mismo esquema que training.
- `pesos`: V de SLS y Vpca, además de la información necesaria para reutilizar la reducción: f, dimensiones, media PCA y normalización inicial o una referencia inequívoca a ella. Su contenido detallado es una propuesta de contrato.
- `pesoftmax`: Wsm, normalización reducida, orden de clases y opción de sesgo. Conservar también hiperparámetros para trazabilidad es recomendable.
- `costo`: historial de la función de costo de Softmax con definición y eje de iteración. El formato no está fijado por [E].

Resultados exigidos sin nombre de archivo prescrito

Gráfica de costo, matriz de confusión y F-scores. Pueden guardarse como imágenes y tablas con nombres acordados por el equipo, sin presentarlos como nombres textuales de [E]. La curva de reconstrucción SLS y la curva de energía PCA son diagnósticos recomendados.

Apoyos recomendados para compartir y reproducir

Un README con comandos, dependencias y versiones; una configuración con semillas y parámetros; un registro de decisiones pendientes/resueltas; y una nota breve que explique datos, partición, dimensiones D->J->r, costo, varianza conservada, métricas y limitaciones. [E] no pide expresamente un informe ni una presentación independiente.

Checklist del resultado final

[ ] Están los cuatro scripts y los siete artefactos con nombre exigido.

[ ] Están la gráfica de costo, la matriz de confusión y F1 de las cuatro clases.

[ ] Las fórmulas, parámetros y decisiones tienen referencia y justificación.

[ ] Los pesos contienen toda la información necesaria para repetir inferencia.

[ ] La evaluación completa corre en el orden indicado desde un entorno limpio.

[ ] Se documentan datos usados, partición, resultados reales y limitaciones, sin prometer un porcentaje de acierto.

12. Parte 7: integración, reparto y cierre

Dependencias: partes 0-6 terminadas. **Responsable sugerido:** integrador del equipo, con revisión cruzada. **Resultado:** paquete listo para entregar y evidencia de que puede ejecutarse de principio a fin.

Reparto sugerido por responsabilidades

- **Datos y MDE:** inventario, ventanas, fórmula oficial MDE, etiquetas, partición y `mdef.py`.
- **Reducción:** costos y gradientes SLS, RMSprop+RLS, PCA, persistencia de `pesos` y `dimr.py`.
- **Clasificación y evaluación:** `train.py`, `test.py`, persistencia de `pesoftmax`, costo y métricas.
- **Integración y revisión:** contratos de archivos, ejecución limpia, revisión matemática cruzada y preparación de la entrega.

Son funciones, no un requisito de tener cuatro integrantes. Si trabajan tres personas, una puede coordinar integración. Quien implemente un bloque debería recibir revisión de otra persona antes de darlo por cerrado. Las interfaces pueden acordarse desde el inicio; los resultados finales dependen de la cadena completa.

Calendario sugerido hacia el 24 de octubre

- **19 de octubre:** resolver datos y materiales faltantes, revisar discrepancias y cerrar interfaces. Preparar el inventario y la configuración.
- **20 de octubre:** completar MDE, etiquetas y partición; revisar alineamiento y fuga de información.
- **21 de octubre:** implementar y verificar SLS, sus gradientes y la solución RLS.
- **22 de octubre:** completar PCA, `pesos`, entrenamiento Softmax y persistencia.
- **23 de octubre:** evaluar el modelo final, revisar métricas y ejecutar integración desde cero.
- **24 de octubre:** revisar paquete, reproducibilidad y nombres; dejar margen antes de las 23:00 para subirlo al aula virtual.

El calendario supone que se obtienen a tiempo la rutina interna `Entropy`, los datos y las decisiones metodológicas. Si esas entradas faltan, deben quedar como bloqueos identificados de implementación; no como tareas supuestamente completadas.

Checklist de la parte 7

[ ] Ejecutar `mdef.py -> dimr.py -> train.py -> test.py` desde datos originales.

[ ] Repetir una ejecución con la misma configuración y revisar reproducibilidad dentro de tolerancias.

[ ] Ejecutar `test.py` en un proceso nuevo con los artefactos persistidos.

[ ] Revisar rutas, dependencias, nombres exactos y disponibilidad de todos los archivos.

[ ] Hacer revisión matemática cruzada y registrar consultas resueltas y pendientes.

[ ] Verificar que resultados y gráficas corresponden al mismo modelo y partición finales.

[ ] Preparar la entrega con los requisitos de [E] y subirla al aula virtual dentro del plazo.

Qué se tendrá al concluir

Una aplicación reproducible que transforma señales reales de cuatro clases en predicciones de salud del rodamiento usando MDE, SLS con RMSprop+RLS, PCA-SVD y Softmax con RMSprop; sus datos procesados y pesos persistidos; la curva de costo y evaluación final; y evidencia suficiente para revisar que cada transformación, derivada y métrica corresponde al método acordado.

En esta etapa solo se ha completado la lectura y el plan. La implementación, los experimentos y la entrega al aula virtual quedan como trabajo futuro organizado por las checklists anteriores.